Représentation des entiers relatifs et des flottants

Nous avons vu précédemment comment écrire les nombres entiers naturels en binaire :

-
-

Nous avons vu aussi que les nombres entiers étaient codés au niveau machine sur un nombre d'octets bien défini :

- sur 1 octet, on code nombres, soit les nombres de à ;
- sur 2 octets, on code nombres, soit les nombres de à (en C, c'est le type `int`) ;
- sur 4 octets, on code nombres (en C, c'est le type `long`) ;
- sur 8 octets, on code nombres (en C, c'est le type `long long`).

Enfin nous avons constaté que les opérations arithmétiques basiques (addition, soustraction, multiplication,...) étaient compatibles avec cette notation.

Il nous faut maintenant nous intéresser aux autres possibilités de nombres : **les nombres signés** (relatifs) et les **nombres flottants**(décimaux).

1. Les Entiers Relatifs (entiers signés)

1.1. Une version naïve

Bit de signe

Sur un octet, le **bit de poids fort** - c'est-à-dire le bit le plus à gauche du nombre, représente le signe (pour positif et pour négatif) et les autres représentent la valeur absolue du nombre.

Exercice

Énoncé

1. Quelle est la représentation de dans ce système ?
2. Quelle est la représentation de dans ce système ?
3. Quel est le plus grand nombre représentable sur un octet ?
4. Quel est le plus petit nombre représentable sur un octet ?
5. Combien de nombres sont alors représentés ?
6. Quel problème pouvez-vous déjà constater ?
7. Effectuer l'addition binaire des nombres et . Le résultat est-il logique ?

Solution

A venir !

1.2. Complément à 2

Notation en complément à 2

Pour remédier aux problèmes soulevés par la version naïve, on utilisera la notation en **complément à deux**. Dans cette notation, sur un octet :

- Les nombres positifs sont représentés comme pour les nombres entiers naturels.
- Pour les nombres négatifs, par contre :
  - On détermine l'écriture binaire de la **valeur absolue du nombre**, généralement sur un octet.
  - On **inverse les bits** de l'écriture binaire , ce qu'on appelle le **complément à 1** (cela correspond à une opération logique `NON` sur chaque bit, donc devient et devient) ;
  - On ajoute au résultat, les dépassements (`overflow`, voir plus bas) étant ignorés - ce qui signifie qu'on reste bien sur le nombre de bits définis dans l'étape 1.

Exemple

- Le nombre est représenté par .
- Pour le nombre :
  - on cherche d'abord la représentation de sa valeur absolue : ;
  - ensuite on inverse chaque bit :
  - puis on ajoute au résultat :

Le nombre a pour représentation en complément à 2 sur un octet la suite

Pour les curieux : « Overflow »

Exercice

Enoncé

1. Compléter le tableau suivant (un calcul par élève) :

é à é à

1. Quel est le bit de poids fort sur chacun des nombres donnés ?
2. Quel relation existe-t-il entre les nombres relatifs et leur entier naturel correspondant ?
3. Quel est le plus grand nombre représentable sur un octet ?
4. Quel est le plus petit nombre représentable sur un octet ?
5. Combien de nombre sont alors représentés ?
6. Effectuer l'addition binaire de et . Est-elle compatible avec l'opération décimale ?

Solution

A venir !

Complément à

Le nombre entier relatif négatif possède le même codage sur bits que le nombre entier naturel .

Remarque

Un algorithme simple pour trouver un complément à 2 de tête est de garder tous les chiffres de la valeur absolue depuis le bit de poids faible jusqu'au premier , puis d'inverser tous les suivants. Par exemple :

- Pour obtenir le code de sur 1 octet :
  - Le nombre s'écrit
  - On garde la partie à droite
  - On inverse la partie à gauche après le premier
  - On a ainsi obtenu le codage du nombre sur 1 octet.
- Pour obtenir le code de sur 1 octet :
  - Le nombre s'écrit
  - On garde uniquement le bit de poids faible
  - On inverse tous les autres bits
  - On a ainsi obtenu le codage du nombre sur 1 octet.

2. Les Flottants

2.1. Représentation de la partie décimale d'un nombre en écriture binaire

Représentation de la partie décimale en base 10

En base 10, on a :

Partie « décimale » en écriture binaire

De la même manière qu'en écriture de base , on aura :

— — — —

Convertir de la base 10 à la base 2

On cherche à convertir en binaire le nombre . On commence par convertir la partie entière . Il faut ensuite convertir la partie décimale . Pour y arriver on applique l'algorithme suivant :

- on multiplie par : , et on note le résultat ;
- on multiplie par : , et on note le résultat ;
- on multiplie par : , et on note le résultat ;
- on multiplie par : , et on note le résultat , la partie décimale étant nulle on arrête l'algorithme ;

On récupère les parties entières obtenues à chaque étape, soit , qui représente la partie décimale de notre nombre. D'où :

Exercice

Enoncé

1. Trouver la représentation décimale de .
2. Trouver la représentation binaire de .
3. Trouver la représentation binaire de . Que remarque-t-on ?

Solution

A venir !

Remarque

Tout comme le nombre — ne possède pas d'écriture décimale finie, certains nombres ne possèdent pas d'écritures binaires finies.

2.2. Écriture scientifique des nombres en écriture binaires

Écriture décimale scientifique

Pour écrire de très grands nombres, ou de très petits, on utilise souvent une écriture scientifique, c'est-à-dire sous la forme , où et . Ainsi :

-
-

Écriture scientifique en binaire

Pour écrire un nombre binaire en « écriture scientifique », on l'écrira sous la forme , où :

- est un nombre binaire, avec partie « après la virgule » et dont la partie entière est ;
- est un nombre entier relatif, **écrit en binaire**.

Exemple

- Pour écrire le nombre en « écriture scientifique », il faudra décaler la virgule de rangs sur la gauche, soit une puissance positive. Or . Donc .
- Pour écrire le nombre en « écriture scientifique », il faudra décaler la virgule de rangs sur la droite, soit une puissance négative, or . Donc .

2.3. Représentation des flottants en machine

Le contenu de cette partie est largement issu de Pixees.fr.

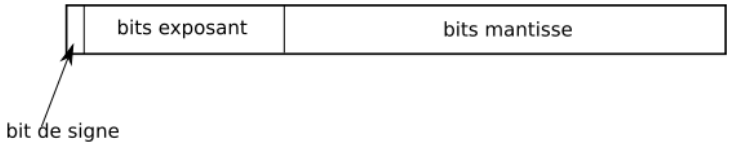
Norme `IEEE 754`

La norme `IEEE 754` est la norme la plus employée pour la représentation des nombres à virgule flottante dans le domaine informatique. La première version de cette norme date de 1985.

Nous allons étudier deux formats associés à cette norme : le format dit **« simple précision »** et le format dit **« double précision »**. Le format « simple précision » utilise bits pour écrire un nombre flottant alors que le format « double » utilise bits. Dans la suite nous travaillerons principalement sur le format bits.

Que cela soit en simple précision ou en double précision, la norme `IEEE 754` utilise :

- 1 bit de signe (1 si le nombre est négatif et 0 si le nombre est positif);
- des bits consacrés à l'**exposant** (8 bits pour la simple précision et 11 bits pour la double précision);
- des bits consacrés à la **mantisse** (23 bits pour la simple précision et 52 bits pour la double précision).



Nous pouvons vérifier que l'on a bien bits pour la simple précision et bits pour la double précision.

Déterminer la mantisse

Pour écrire un nombre au format `IEEE 754`, il est nécessaire de commencer par écrire notre nombre sous « écriture scientifique » binaire, sous la forme .

Ainsi on sait déjà que le nombre .

La partie « » du nombre, c'est-à-dire , correspond à la **mantisse**. Mais comme la mantisse doit contenir exactement bits en simple précision, il faut donc la compléter avec zéros à droite pour obtenir bits, soit :

Exposant des nombres flottants

Notre première intuition serait de dire que la partie *exposant* correspond simplement au « e » de (dans notre exemple , nous aurions). En fait, c'est un peu plus compliqué... En effet, comment représenter les exposants négatifs, sachant que dans la norme `IEEE 754`, aucun bit pour le signe de l'exposant n'a été prévu. Il a donc été nécessaire de choisir une méthode, et c'est celle du **décalage d'exposant** qui a été retenue :

- en simple précision, on décale l'exposant de ;
- en double précision, on décale l'exposant de .

Déterminer l'exposant

Pour le format simple précision, bits sont consacrés à l'exposant, il est donc possible de représenter valeurs, nous allons pouvoir représenter des exposants compris entre et (les valeurs et sont des valeurs réservées, nous n'aborderons pas ce sujet ici, mais vous trouverez des précisions sur l'article IEEE 754 de wikipedia}). Pour avoir des valeurs uniquement positives, il va falloir procéder à un **décalage** : ajouter systématiquement à la valeur de l'exposant.

Dans notre exemple, on a . L'exposant est donc , et pour le représenter il faut donc le **décaler** : soit . L'exposant possède 8 chiffres binaires, il n'est donc pas nécessaire de rajouter de zéros **à gauche**.\

Au format simple précision, le nombre s'écrit donc :

Exemple

Soit le nombre « » en base . Représentons-le au format simple précision :

- nous avons et soit ;
- décalons la virgule : = , soit avec le décalage de l'exposant , en écrivant l'exposant en base 2, nous obtenons ;
- nous avons donc : notre bit de signe = (nombre négatif), nos 8 bits d'exposant = et nos 23 bits de mantisse =
- soit en "collant" tous les "morceaux" :

Exercice

Énoncé

1. Déterminez la représentation au format simple précision de en binaire.
2. Déterminez la représentation au format simple précision de en binaire.
3. Soit le nombre flottant au format simple précision : . Trouvez la représentation en base 10 de ce nombre.
4. À l'aide de Thonny, tapez dans la console : . Faites de même dans la console Javascript de Firefox. Que constate-t-on ? Pourquoi ?

Solution

A venir !

œ